Digital Visitor Management System

Prototype Application and Implementation Report

Contribution Part: Realisation

Prepared by:

Serrar Lokmane
Laouar Younes
Bouakaken Sif Eddine

Academic Year: 2025-2026
Department of Computer Science and Software Engineering
May 02, 2026

Contents

Abstract

This report presents the realisation phase of the Digital Visitor Management System proposed in the previous design report. The application is implemented as a Python web prototype using Flask for the presentation and application layers and SQLite for persistent storage. Its objective is to replace the manual paper-based visitor log used at the reception service with a structured digital workflow that supports registration, search, exit tracking, reporting, and export.

The prototype follows the UML design defined in Rapport 2. It implements the main Visitor entity and the visitor management operations: adding a visitor, searching records, recording departures, generating statistics, and exporting data. The report also describes the tools used, presents the application interfaces, and includes representative source code extracts.

1 Introduction

The first report identified the operational problem at the reception desk: visitor information is recorded manually in paper logbooks, which creates delays, weak traceability, poor data reliability, and limited reporting. The second report proposed a digital visitor management system using UML and a three-tier architecture.

This third report completes the project cycle by implementing a working prototype. The application is intentionally simple and focused on the core needs of reception staff: registering visitors quickly, storing records reliably, locating visitor history, tracking current occupancy, and providing basic management indicators.

2 Development Tools and Environment

The prototype was developed with a small and portable Python stack. This choice makes the application easy to run on a local workstation without requiring a complex server installation.

Tool	Role in the prototype
Python 3	Main programming language used to implement the application logic.
Flask	Lightweight web framework used for routing, templates, forms, and responses.
SQLite	Embedded relational database used to store visitor records locally.
HTML/CSS	Presentation layer used to create the dashboard, forms, tables, and reports.
ReportLab	Python library used to generate this PDF report.

3 Prototype Application Overview

The application is organized around four main screens: a reception dashboard, a visitor registration form, a searchable records page, and a reporting page. The dashboard summarizes current activity. The registration form captures identity and visit data. The records screen supports search, filtering, exit tracking, and CSV export. The reports page provides daily and departmental statistics.

The interface was designed for repeated operational use: restrained colors, compact tables, clear buttons, and stable page layouts. The goal is to help reception staff complete tasks quickly while giving administrators a direct overview of visitor activity.

4 Database and Application Logic

The database contains one main table named visitors. Each record includes a unique identifier, visitor name, phone number, purpose, target department, entry time, optional exit time, student status, university, and ID card number. The exit time remains empty while the visitor is still inside the building.

The VisitorStore class centralizes all database operations. This separation keeps SQL operations outside the route functions and follows the VisitorManager concept described in the UML model. The Flask routes call this class to add records, retrieve filtered lists, update exit times, calculate statistics, and export CSV files.

5 Application Interfaces

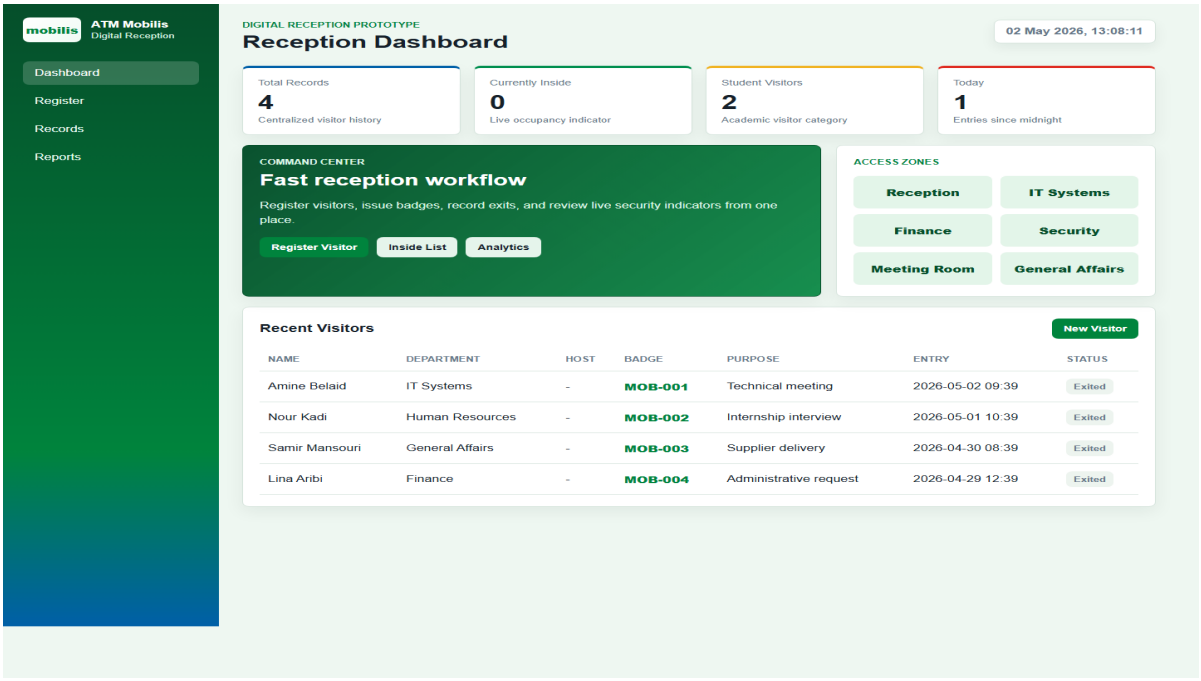

Figure 1: Reception dashboard with current visitor indicators and recent records.

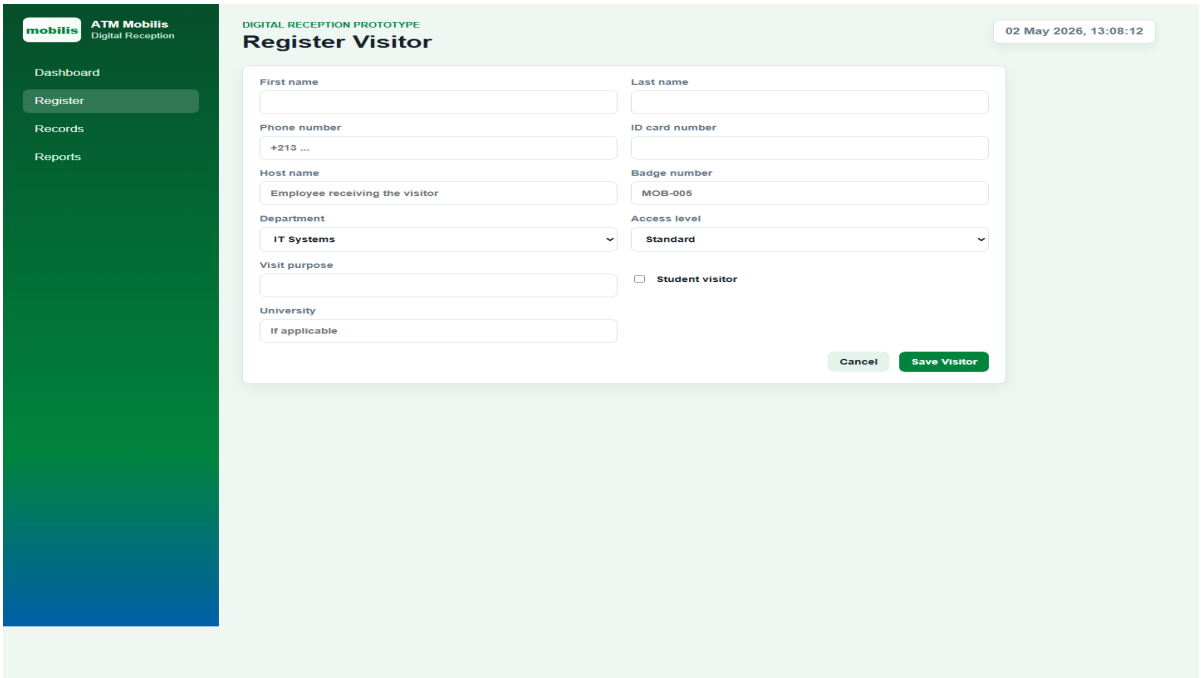

Figure 2: Visitor registration interface with identity, purpose, department, and student fields.

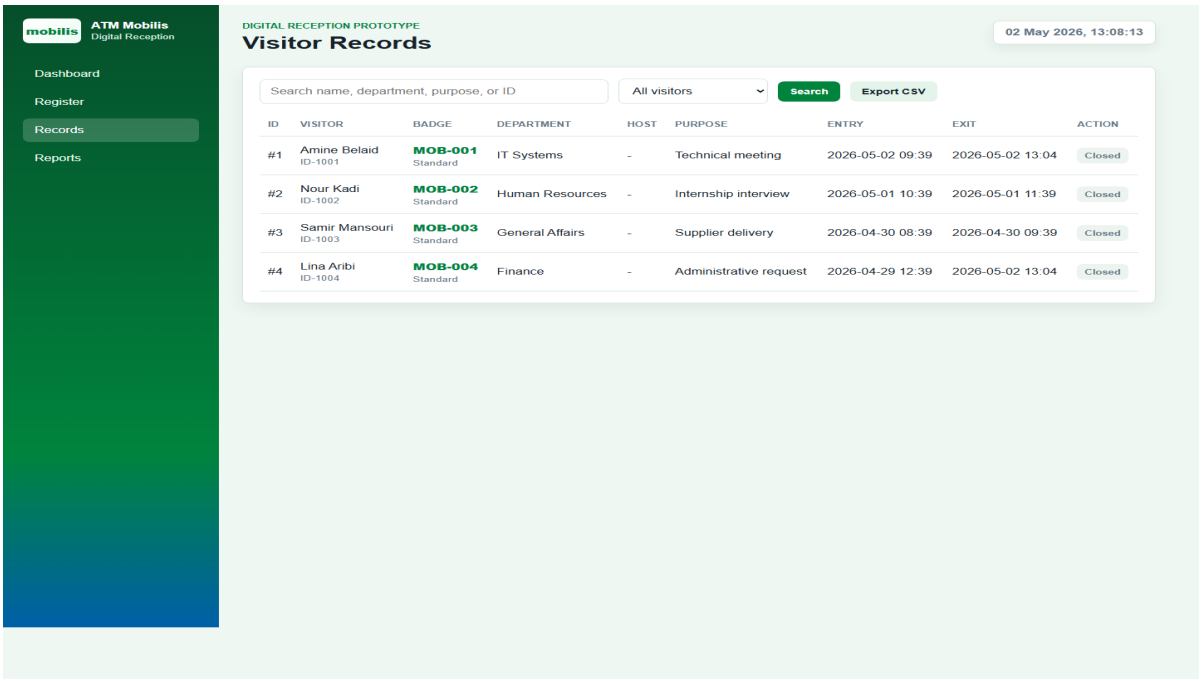

Figure 3: Searchable records table with visitor status and exit action.

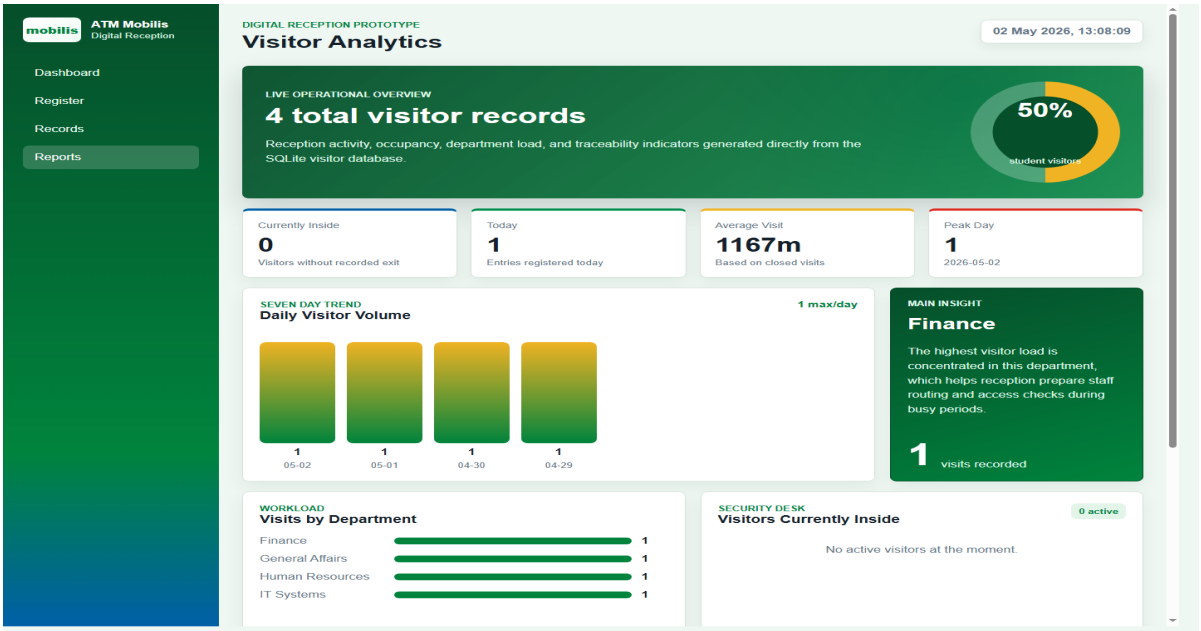

Figure 4: Reporting page showing daily and department-based visitor statistics.

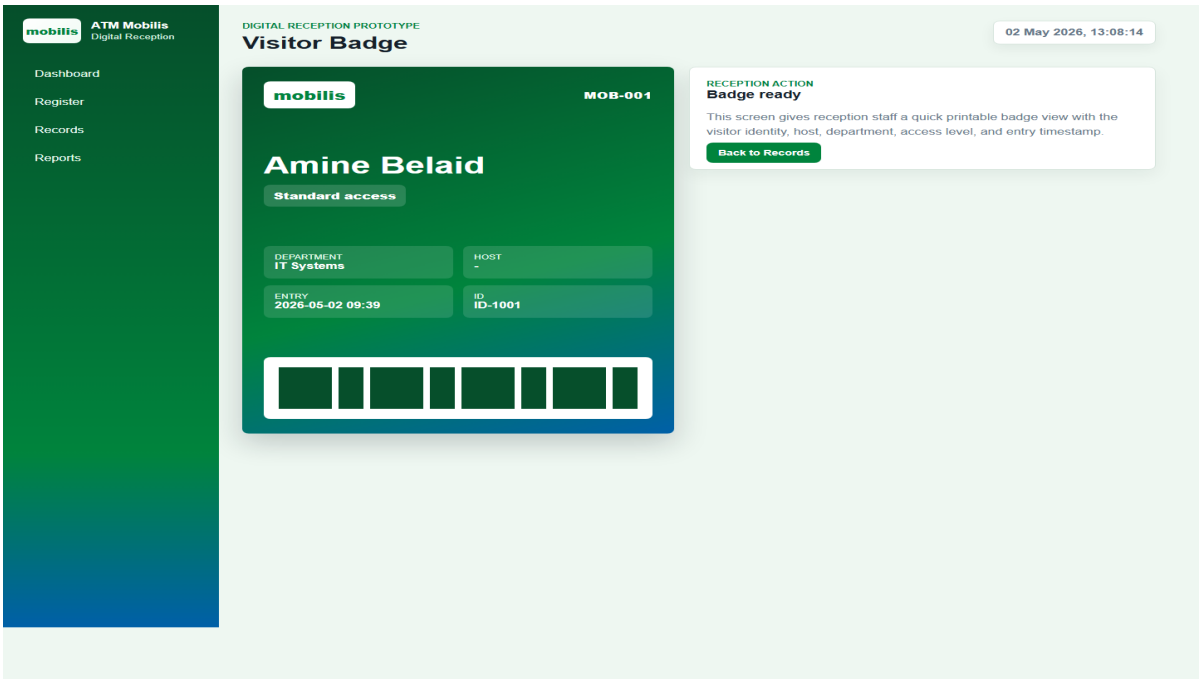

Figure 5: Visitor badge preview with Mobilis branding, host, access level, and entry details.

6 Source Code Extracts

The following extracts show the principal implementation elements. The complete source code is included in the project folder.

6.1 Flask Routes

```
from __future__ import annotations

from datetime import datetime
from pathlib import Path

from flask import Flask, flash, redirect, render_template, request, send_file, url_for

from visitor_store import VisitorStore


BASE_DIR = Path(__file__).resolve().parent
DB_PATH = BASE_DIR / "data" / "visitors.db"

app = Flask(__name__)
app.config["SECRET_KEY"] = "visitor-management-prototype"
store = VisitorStore(DB_PATH)


@app.context_processor
def inject_now():
    return {"now": datetime.now}


@app.route("/")
def dashboard():
    stats = store.dashboard_stats()
    recent_visitors = store.list_visitors(limit=8)
    return render_template("dashboard.html", stats=stats, recent_visitors=recent_visitors)


@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        payload = {
            "first_name": request.form.get("first_name", "").strip(),
            "last_name": request.form.get("last_name", "").strip(),
            "phone_number": request.form.get("phone_number", "").strip(),
            "visit_purpose": request.form.get("visit_purpose", "").strip(),
            "department": request.form.get("department", "").strip(),
            "host_name": request.form.get("host_name", "").strip(),
            "badge_number": request.form.get("badge_number", "").strip(),
            "access_level": request.form.get("access_level", "Standard").strip(),
            "is_student": 1 if request.form.get("is_student") == "on" else 0,
            "university": request.form.get("university", "").strip(),
            "id_card_number": request.form.get("id_card_number", "").strip(),
        }
        errors = validate_visitor(payload)
        if errors:
            for error in errors:
                flash(error, "error")
            return render_template("register.html", form=payload)

        visitor_id = store.add_visitor(payload)
        flash(f"Visitor #{visitor_id} registered successfully.", "success")
        return redirect(url_for("records"))

    return render_template("register.html", form={})


@app.route("/records")
def records():
    query = request.args.get("q", "").strip()
    status = request.args.get("status", "all")
    visitors = store.search_visitors(query=query, status=status)
    return render_template("records.html", visitors=visitors, query=query, status=status)
```

6.2 SQLite Data Access

```
from __future__ import annotations

import csv
import sqlite3
from datetime import datetime, timedelta
from pathlib import Path
from typing import Any


class VisitorStore:
    def __init__(self, db_path: Path):
        self.db_path = Path(db_path)
        self.db_path.parent.mkdir(parents=True, exist_ok=True)
        self.initialize()

    def connect(self) -> sqlite3.Connection:
        connection = sqlite3.connect(self.db_path)
        connection.row_factory = sqlite3.Row
        return connection

    def initialize(self) -> None:
        with self.connect() as connection:
            connection.execute(
                """
                CREATE TABLE IF NOT EXISTS visitors (
                    visitor_id INTEGER PRIMARY KEY AUTOINCREMENT,
                    first_name TEXT NOT NULL,
                    last_name TEXT NOT NULL,
                    phone_number TEXT,
                    visit_purpose TEXT NOT NULL,
                    department TEXT NOT NULL,
                    entry_time TEXT NOT NULL,
                    exit_time TEXT,
                    is_student INTEGER NOT NULL DEFAULT 0,
                    university TEXT,
                    id_card_number TEXT NOT NULL,
                    host_name TEXT,
                    badge_number TEXT,
                    access_level TEXT NOT NULL DEFAULT 'Standard'
                )
                """
            )
            self._migrate(connection)
            count = connection.execute("SELECT COUNT(*) FROM visitors").fetchone()[0]
            if count == 0:
                self._seed(connection)

    def _migrate(self, connection: sqlite3.Connection) -> None:
        columns = {row["name"] for row in connection.execute("PRAGMA table_info(visitors)")}
        migrations = {
            "host_name": "ALTER TABLE visitors ADD COLUMN host_name TEXT",
            "badge_number": "ALTER TABLE visitors ADD COLUMN badge_number TEXT",
            "access_level": "ALTER TABLE visitors ADD COLUMN access_level TEXT NOT NULL DEFAULT 'Standard
        }
        for column, sql in migrations.items():
            if column not in columns:
                connection.execute(sql)

    def _seed(self, connection: sqlite3.Connection) -> None:
        now = datetime.now()
        seed_rows = [
            ("Amine", "Belaid", "+213 550 100 100", "Technical meeting", "IT Systems", now - timedelta(ho
            ("Nour", "Kadi", "+213 661 234 987", "Internship interview", "Human Resources", now - timedel
            ("Samir", "Mansouri", "+213 770 808 111", "Supplier delivery", "General Affairs", now - timed
            ("Lina", "Aribi", "+213 555 909 202", "Administrative request", "Finance", now - timedelta(da
```

7 Testing and Validation

The prototype was tested by launching the Flask server locally, opening each main interface, verifying that seed records were loaded, registering a new visitor, checking that the record appeared in the table, and confirming that the reporting page updated its counters from the database.

Validation rules were also checked on the registration form. Required fields prevent incomplete records, phone numbers are constrained to simple numeric formats, and the university field becomes mandatory when a visitor is marked as a student. These controls reduce the risk of missing or inconsistent data compared with the manual paper log.

8 Conclusion

The realised prototype demonstrates that the proposed digital visitor management system can solve the main reception problems identified during the internship: slow manual registration, lack of searchability, weak traceability, and absence of basic statistics. By using Python, Flask, and SQLite, the solution remains simple, understandable, and portable while still implementing the core workflow required by reception staff.

Future improvements may include user authentication, role-based access control, badge printing, appointment pre-registration, cloud deployment, and integration with physical access control devices. Even in its current prototype form, the application provides a practical foundation for replacing the paper register with a structured digital system.